PROYECTO FINAL

CRM de ventas

Un núcleo común, el sector que ustedes elijan

Programación Web

Ingeniería en Sistemas · Sexto semestre · 2026
Universidad Mesoamericana · Facultad de Ingeniería · Quetzaltenango
Ing. Fredy Ramírez Solano · ronyramsol@umes.edu.gt

Proyecto en parejas. Una sola entrega, con defensa oral de 15 minutos por pareja.

> **En pocas palabras**
>
> En parejas, ustedes construirán un CRM: el sistema con el que una empresa controla a sus clientes, sus oportunidades de venta y lo que vendió cada mes. Ustedes eligen la empresa y su sector; el núcleo de reglas es el mismo para todas las parejas.
>
> El sistema tiene tres roles y dos puertas sobre una sola base MySQL: el administrador y el supervisor trabajan en un panel hecho con Laravel; el vendedor trabaja en una SPA que consume una API REST propia, construida con Node.js y Express.
>
> El proyecto se desarrolla guiado por especificaciones, con asistencia de inteligencia artificial, y se sustenta en una defensa oral.

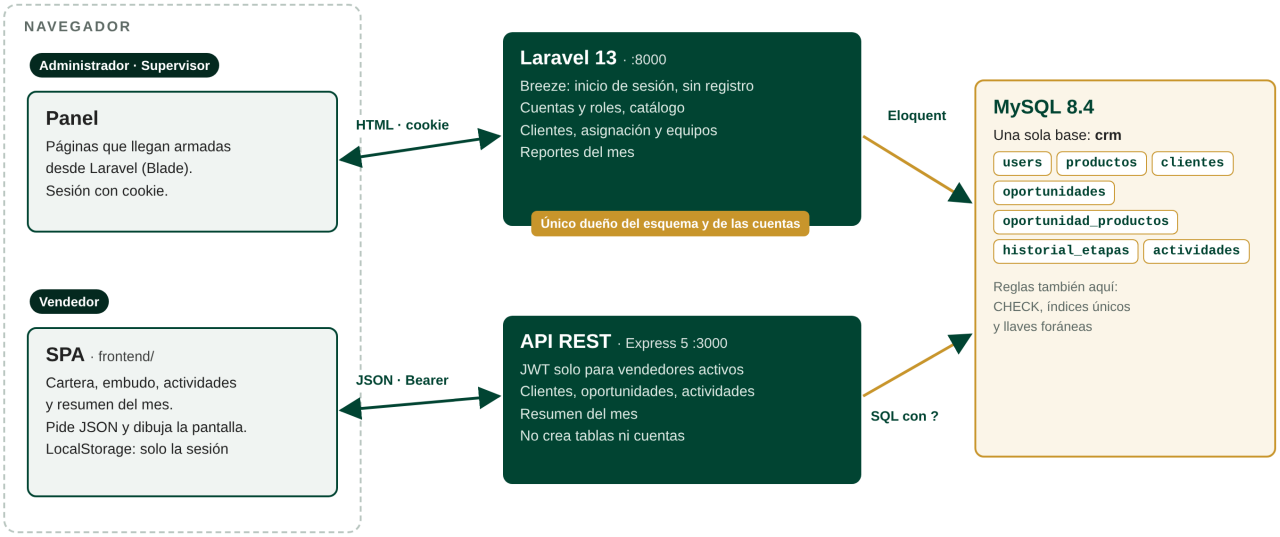
1. El problema

Muchas empresas pequeñas llevan a sus clientes en hojas de cálculo y cuadernos. Cuando el negocio crece, aparecen los mismos problemas en cualquier sector:

- Dos vendedores llaman al mismo cliente y le ofrecen precios distintos.
- Nadie sabe cuánto se vendió en el mes hasta que alguien suma a mano, y cada quien suma distinto.
- El gerente no sabe qué ventas están por cerrarse ni cuánto puede esperar del mes.
- Un vendedor se va de la empresa y con él se van sus clientes y lo que les había cotizado.

Un CRM ordena eso: cada cliente tiene un vendedor responsable, cada venta posible es una **oportunidad** que avanza por etapas, los precios cotizados quedan registrados y los reportes salen de la base de datos, no de una suma a mano.

2. Qué van a construir



Pieza	Quién la usa	Qué hace	Lo que la distingue
Panel Laravel	Administrador y supervisor	Cuentas, catálogo, clientes y su asignación, oportunidades del equipo y reportes del mes	Arquitectura monolítica: Blade arma las páginas en el servidor. Es la única dueña del esquema y la única que crea cuentas
API REST	La SPA del vendedor	Cartera de clientes, oportunidades con sus productos, actividades de seguimiento y resumen del mes, en JSON	Servicio desacoplado con JWT. Lee y escribe sobre las tablas que definió Laravel, con SQL escrito a mano
SPA	Vendedor	La aplicación diaria del vendedor: su cartera, su embudo de ventas, sus pendientes y su resumen	Cliente en el navegador, sin framework. LocalStorage solo guarda la sesión

Las dos puertas trabajan sobre los mismos datos. Un cliente que el supervisor asigna desde el panel aparece en la cartera del vendedor en la SPA; una venta que el vendedor gana en la SPA aparece en el reporte del panel. Esa es la prueba central del proyecto.

Dos reglas de arquitectura que no se negocian

- **Un solo dueño del esquema.** Las tablas se crean y se modifican únicamente con migraciones de Laravel. La API nunca ejecuta `CREATE` ni `ALTER`.
- **Un solo creador de cuentas.** Las cuentas las crea el administrador desde el panel. No hay registro público en ninguna de las dos puertas, y la API solo verifica contraseñas.

Por qué cada regla es necesaria forma parte de lo que ustedes deben poder explicar en la defensa.

3. Núcleo común y capa de sector

El proyecto tiene dos partes con reglas distintas:

	Núcleo común	Capa de sector
Qué es	Roles, modelo de datos, reglas del negocio, contrato de la API y semilla de verificación: todo lo que describe este enunciado	La adaptación del CRM a la empresa y al sector que ustedes eligieron
Para quién	Igual para todas las parejas	Distinta para cada pareja
Quién decide	Este enunciado	Ustedes, en su especificación
Cómo se verifica	Con pruebas automáticas del catedrático, que no se publican, contra la semilla de verificación	Con su especificación, sus pruebas y la defensa

Elegir el sector

Cada pareja elige una empresa de un sector concreto: una ferretería, una clínica dental, una distribuidora de agua purificada, una academia de idiomas, un taller de motos, una empresa de paneles solares. Dentro de una misma sección no se repiten sectores: registren el suyo en el aula virtual y, si otra pareja de su sección ya lo registró, elijan otro.

Qué incluye la capa de sector

- **El vocabulario de las pantallas.** Una clínica puede decir "pacientes" y "tratamientos" donde el núcleo dice clientes y productos. El cambio es de textos en el panel y en la SPA: las rutas, los campos del contrato y las tablas del núcleo no cambian.
- **Un `SectorSeeder` con datos creíbles de su sector:** un catálogo con precios del mercado guatemalteco, supervisores y vendedores propios, clientes, oportunidades en todas las etapas y actividades. Corre después de la semilla de verificación: no puede repetir sus correos, NIT ni teléfonos, ni pasar de 10 clientes por vendedor.
- **Una regla de negocio propia**, que no esté en este enunciado y que tenga sentido en su sector. Debe cumplirse en el servidor (y en la base de datos si aplica), tener criterios de aceptación en su especificación y una prueba que la demuestre.
- **Si la necesitan, una columna o tabla adicional**, siempre que no rompa el contrato: una columna nueva en una tabla del núcleo debe aceptar vacío o tener un valor por omisión.

Sector	Ejemplo de regla propia
Paneles solares	Una oportunidad que incluye paneles no puede ganarse sin el servicio de instalación en la misma oportunidad
Clínica dental	Un tratamiento de ortodoncia no pasa a propuesta sin una actividad de evaluación completada con ese paciente
Distribuidora	A un cliente con NIT de empresa (no CF) no se le gana un pedido por debajo de un monto mínimo
Academia de idiomas	Un estudiante no puede tener abiertas dos oportunidades del mismo nivel al mismo tiempo

> **El límite de la capa de sector**
>
> Con solo la semilla de verificación cargada, su sistema debe comportarse exactamente como dice este enunciado. Su regla propia no puede impedir lo que el núcleo exige ni cambiar las cifras de control. Si su regla choca con el núcleo, la regla está mal planteada: replantéenla en su especificación.

4. Reglas del proyecto

- **Parejas, nota individual.** Se trabaja en parejas, pero la nota del proyecto se asigna a cada integrante según lo que explique y sustente en la defensa. Cualquiera de los dos responde por cualquier parte del código.
- **Agente de IA libre.** Pueden usar el agente que prefieran (Claude Code, OpenCode, Codex, Cursor u otro). Todo el código entregado es responsabilidad de la pareja, lo haya escrito una persona o un agente.
- **Git obligatorio.** El proyecto vive en un repositorio de GitHub o GitLab con commits de ambos integrantes a lo largo del proyecto. Se entrega el enlace, no un archivo comprimido.
- **Eloquent en el panel, SQL en la API.** Laravel usa Eloquent. La API no usa ORM ni constructor de consultas (nada de Sequelize, Prisma, TypeORM o Knex): sus consultas se escriben en SQL con `mysql2` y parámetros.
- **La SPA no usa framework:** HTML, Bootstrap y JavaScript con módulos.

Entorno y versiones

Herramienta	Versión	Observación
PHP y Composer	PHP 8.4	Laravel 13 exige PHP 8.3 o superior
Laravel	13, con Breeze (stack Blade)	Breeze se instala aparte; vea la nota
Base de datos	MySQL 8.4 LTS	Las restricciones CHECK son parte del proyecto, y MySQL 8.4 las aplica
Node.js	24 LTS	
API	Express 5, mysql2, jsonwebtoken, bcryptjs y cors	Ningún otro paquete es obligatorio
SPA	Bootstrap 5.3 y módulos de JavaScript	Se sirve con Live Server o con cualquier servidor estático
Pruebas de la API	Postman	Su colección se entrega en el repositorio

> **Trampas de instalación**
>
> **Breeze no aparece en el asistente de `laravel new`.** Cree el proyecto sin kit de inicio y luego ejecute `composer require laravel/breeze --dev` y `php artisan breeze:install blade`.
>
> **Laravel usa SQLite por omisión.** Si no cambia `DB_CONNECTION` y los datos de conexión en `.env`, el panel y la API trabajarán sobre bases distintas sin que nada falle a la vista.
>
> **Los estilos de Breeze se compilan.** Si agrega clases en sus vistas y no aparecen, falta `npm run dev` o `npm run build`.

5. Roles y puertas

Rol	Puerta	Qué puede hacer	Qué no puede hacer
Administrador	Panel	Crear y editar cuentas; administrar el catálogo; ver y administrar todos los clientes; ver todas las oportunidades y los reportes de toda la empresa	Entrar a la API. Borrar cuentas: se desactivan
Supervisor	Panel	Ver a los vendedores de su equipo, los clientes de su equipo y los disponibles; crear clientes y asignarlos, reasignarlos o liberarlos dentro de su equipo; ver las oportunidades y los reportes de su equipo	Ver otros equipos. Administrar cuentas o catálogo. Entrar a la API
Vendedor	SPA, a través de la API	Trabajar su cartera, tomar clientes disponibles, llevar sus oportunidades y sus productos, registrar actividades y ver su resumen del mes	Entrar al panel. Ver o tocar datos de otro vendedor

Cada vendedor pertenece al equipo de un supervisor. Una cuenta desactivada no entra por ninguna de las dos puertas y, si tenía una sesión abierta, la pierde en su siguiente petición.

6. Modelo de datos

Las tablas son las mismas para todas las parejas. Los tipos de la columna "Tipo" son los de MySQL; las migraciones los expresan con los métodos de Laravel. Todas las fechas se guardan en UTC.

users

La tabla que crea Laravel, más tres columnas que agrega una migración propia:

Columna	Tipo	Reglas
`rol`	administrador, supervisor o vendedor	Obligatorio; por omisión, vendedor
`supervisor_id`	Referencia a `users.id`, o vacío	Obligatorio para los vendedores y vacío para los demás. Debe ser una cuenta con rol supervisor
`activo`	Booleano	Por omisión, verdadero. Las cuentas no se borran: se desactivan

productos

Columna	Tipo	Reglas
`nombre`	VARCHAR(120)	Obligatorio y único
`precio`	DECIMAL(10,2)	Mayor que cero, en quetzales
`activo`	Booleano	Solo los activos se agregan a una oportunidad. Los productos no se borran: se desactivan

clientes

Columna	Tipo	Reglas
`vendedor_id`	Referencia a `users.id`, o vacío	El vendedor responsable. Vacío significa que el cliente está disponible
`nombre`	VARCHAR(150)	Obligatorio
`nit`	VARCHAR(12), o vacío	NIT válido guardado compacto (`4521817K`), o vacío para consumidor final. Único
`telefono`	CHAR(8)	Teléfono de Guatemala, solo los 8 dígitos. Único
`correo`	VARCHAR(150), o vacío	Formato de correo válido

oportunidades

Columna	Tipo	Reglas
`cliente_id`	Referencia a `clientes.id`	Obligatorio
`vendedor_id`	Referencia a `users.id`	El dueño de la oportunidad
`titulo`	VARCHAR(150)	Obligatorio
`etapa`	prospecto, calificado, propuesta, negociacion, ganada o perdida	Por omisión, prospecto
`motivo_perdida`	VARCHAR(255), o vacío	Presente si y solo si la etapa es perdida
`fecha_cierre`	DATETIME, o vacío	Presente si y solo si la etapa es ganada o perdida

oportunidad_productos

Cada fila es una línea de la oportunidad: un producto, su cantidad y el precio al que se cotizó.

Columna	Tipo	Reglas
`oportunidad_id`	Referencia a `oportunidades.id`	Al borrar la oportunidad se borran sus líneas

Columna	Tipo	Reglas
`producto_id`	Referencia a `productos.id`	Un producto aparece una sola vez en cada oportunidad (índice único)
`cantidad`	SMALLINT	De 1 a 999
`precio_unitario`	DECIMAL(10,2)	El precio del catálogo en el momento de agregar el producto. Mayor que cero

historial_etapas

Columna	Tipo	Reglas
`oportunidad_id`	Referencia a `oportunidades.id`	Al borrar la oportunidad se borra su historial
`usuario_id`	Referencia a `users.id`	Quién hizo el cambio
`etapa_anterior`	Una etapa, o vacío	Vacío en el registro de creación
`etapa_nueva`	Una etapa	Obligatoria
`created_at`	DATETIME	Cuándo ocurrió el cambio

actividades

Columna	Tipo	Reglas
`cliente_id`	Referencia a `clientes.id`	Al borrar el cliente se borran sus actividades
`vendedor_id`	Referencia a `users.id`	Quién la registró
`tipo`	llamada, reunion o tarea	Obligatorio
`descripcion`	VARCHAR(500)	Obligatoria
`vence_en`	DATETIME, o vacío	Cuándo vence
`completada`	Booleano	Por omisión, falso

Todas las tablas, salvo `historial_etapas`, tienen `created_at` y `updated_at`.

> **Las reglas también viven en la base**
> Dos aplicaciones escriben en estas tablas. Una regla que solo vive en una de ellas no protege nada. Por eso, además de validar en el panel y en la API, la base hace cumplir lo que puede: llaves foráneas, índices únicos (NIT, teléfono, producto por oportunidad) y restricciones `CHECK` (precios y cantidades, formato del teléfono, y la relación entre la etapa, el motivo de pérdida y la fecha de cierre).

7. Reglas del núcleo

Estas reglas son las que verifican las pruebas automáticas. Cada una se cumple en todas las puertas donde aplica.

R1 · Acceso por rol

- Cada rol entra solo por su puerta (sección 5). Un vendedor que intenta usar el panel, o un administrador o supervisor que intenta usar la API, recibe un rechazo claro.
- Una cuenta desactivada no entra por ninguna puerta, y lo que ya tenía abierto deja de servirle: su sesión del panel o su token de la API.

R2 · Propiedad

- Un vendedor solo ve y modifica sus clientes, sus oportunidades y las actividades de los clientes de su cartera. Lo ajeno responde 404, no 403: la respuesta no confirma que exista.
- Un supervisor solo ve a su equipo y a los clientes disponibles.
- El dueño de un registro nunca viene del cuerpo de la petición: sale de la sesión o del token.

R3 · Cartera

- Un vendedor tiene como máximo **10 clientes**. Crear, tomar o asignar un cliente a un vendedor con la cartera llena se rechaza (409 en la API).
- Un cliente sin vendedor está **disponible**. Cualquier vendedor con espacio en su cartera puede tomarlo. La lista de disponibles muestra solo el id y el nombre: sin NIT, teléfono ni correo.
- Tomar un cliente es atómico: si dos vendedores lo intentan al mismo tiempo, exactamente uno lo consigue y el otro recibe 409. Del mismo modo, la cartera máxima se respeta aunque lleguen varias operaciones simultáneas del mismo vendedor.

R4 · Identidad del cliente

NIT. Se acepta un NIT de Guatemala válido o `CF` (consumidor final). Antes de validar, el NIT se compacta: se quitan espacios y guiones, se pasa a mayúsculas y se quitan los ceros a la izquierda. El último carácter es un dígito verificador (0 a 9, o K) que se calcula con módulo 11:

1. Separe el cuerpo (todo menos el último carácter) del verificador.
2. Multiplique cada dígito del cuerpo, de derecha a izquierda, por 2, 3, 4… y sume los productos.
3. Calcule (11 − (suma mód 11)) mód 11. Si da 10, el verificador es K; si no, es ese número.

Ejemplo: `452 1817-k` se compacta a `4521817K`. Cuerpo 4521817: 7×2 + 1×3 + 8×4 + 1×5 + 2×6 + 5×7 + 4×8 = 133; 133 mód 11 = 1; 11 − 1 = 10, que es K. Válido. Un NIT compacto tiene entre 2 y 12 caracteres.

CF se guarda como vacío (`NULL`) y la API lo devuelve como `"CF"`. Así varios clientes pueden ser consumidor final sin chocar con el índice único del NIT.

Teléfono. Se aceptan espacios, guiones, puntos, paréntesis y el prefijo `+502` o `502`; todo eso se quita antes de guardar. Lo que queda deben ser 8 dígitos que empiecen con 2 a 9.

Sin repetidos. El NIT (cuando no es CF) y el teléfono no se repiten entre clientes, comparados ya normalizados: `4521817-k` y `4521817K` son el mismo NIT. Un repetido responde 409 en la API.

R5 · Embudo de ventas

Etapa	prospecto	calificado	propuesta	negociacion	ganada	perdida
Probabilidad	10 %	25 %	50 %	75 %	100 %	0 %

- Toda oportunidad nace en prospecto, para un cliente de la cartera de quien la crea. Las cuatro primeras etapas están **abiertas**; ganada y perdida están **cerradas**.
- Como mínimo, siempre se puede avanzar una etapa a la vez entre las abiertas, ganar desde negociación y perder desde negociación. Los saltos, los retrocesos y desde qué otras etapas se gana o se pierde los decide su especificación.
- Ganar exige al menos un producto en la oportunidad. Perder exige un motivo (de 1 a 255 caracteres, sin contar espacios sobrantes).

- Al cerrarse, la oportunidad registra su fecha de cierre. Una oportunidad cerrada ya no cambia: ni título, ni etapa, ni productos (409).
- El historial registra la creación (con etapa anterior vacía) y cada cambio de etapa, con quién y cuándo, en la misma transacción que el cambio. Un intento rechazado no deja registro, y dos cambios simultáneos sobre la misma oportunidad no pueden dejar dos registros.

R6 · Dinero

- Los montos son quetzales con dos decimales y se guardan como DECIMAL, nunca como FLOAT.
- Al agregar un producto a una oportunidad, su precio se **copia** del catálogo a la línea. Si después cambia el precio del catálogo, la oportunidad no cambia. Quien hace la petición nunca envía el precio.
- El total de una oportunidad es la suma de cantidad × precio unitario de sus líneas, y lo calcula el servidor.
- Solo se agregan productos activos. Un producto que ya está en la oportunidad no se agrega otra vez (409): se cambia su cantidad, que es un entero de 1 a 999.

R7 · Reportes

- Una **venta** es una oportunidad ganada. Su mes es el mes calendario de su fecha de cierre **en hora de Guatemala** (UTC−6, sin horario de verano), aunque la base guarde UTC.
- Total vendido: la suma de los totales de las ventas del mes.
- Pronóstico: la suma de total × probabilidad de las oportunidades abiertas, sin filtro de mes, redondeada a dos decimales.
- Los reportes se calculan en la base de datos (SQL en la API; Eloquent o el constructor de consultas en el panel) y la API los devuelve como números, no como texto.

R8 · Respuestas y fechas

- La API nunca devuelve hashes de contraseñas, llaves internas como `vendedor_id`, `user_id` o `supervisor_id`, ni datos de otro vendedor.
- Las fechas viajan en ISO 8601 y en UTC (`2026-10-05T16:00:00.000Z`). La SPA las muestra en hora de Guatemala.
- Una fecha que llega a la API sin zona horaria (`2026-12-01T10:00`) es ambigua y se rechaza con 422. Con zona (`2026-12-01T10:00:00-06:00`) se guarda en UTC.
- Una actividad está **vencida** si no está completada y su fecha de vencimiento ya pasó.

Decisiones que su especificación debe tomar

El enunciado deja abiertas estas preguntas a propósito. Su especificación las responde, con la alternativa que descartaron y por qué. Lo que decidan debe respetar las reglas de arriba.

Pregunta	Lo que deben considerar
¿Qué transiciones del embudo permiten, además del mínimo?	Saltos de etapa, retrocesos, perder o ganar desde otras etapas, y qué responde la API a lo que no permiten
¿Qué pasa con las oportunidades y actividades de un cliente reasignado?	Quién conserva el crédito de las ventas ya cerradas y quién sigue las abiertas
¿Se puede liberar un cliente con oportunidades abiertas?	Qué pasa con esas oportunidades si el cliente queda disponible
¿Qué pasa con la cartera de un vendedor desactivado?	Si sus clientes quedan asignados, se liberan o esperan a ser reasignados

Pregunta	Lo que deben considerar
¿Qué hace la API con campos que el contrato no lista?	Ignorarlos o rechazarlos con 422; nunca usarlos
¿Cuál es su regla propia?	Por qué tiene sentido en su sector, dónde se cumple y cómo se prueba

8. Contrato de la API

Base: `http://localhost:3000/api`. Todo viaja en JSON. Todas las rutas exigen `Authorization: Bearer <token>`, salvo el inicio de sesión. Pueden agregar endpoints, pero no cambiar estos: el catedrático los verifica con pruebas automáticas.

Método	Ruta	Cuerpo o parámetros	Respuesta correcta	Errores
POST	`/auth/login`	`{ correo, clave }`	200 `{ token, usuario }`	400, 401, 403, 422
GET	`/productos`		200, productos activos	401
GET	`/clientes`	`?q=` (contiene, en nombre o NIT)	200, clientes de su cartera	401
POST	`/clientes`	`{ nombre, nit, telefono, correo }`; correo opcional	201, el cliente, ya en su cartera	400, 401, 409, 422
GET	`/clientes/disponibles`		200, `[{ id, nombre }]`	401
POST	`/clientes/:id/tomar`		200, el cliente, ya en su cartera	401, 404, 409
GET	`/clientes/:id`		200, el cliente	401, 404
PATCH	`/clientes/:id`	Los campos que cambian	200, el cliente	400, 401, 404, 409, 422
POST	`/clientes/:id/actividades`	`{ tipo, descripcion, vence_en }`; vence_en opcional	201, la actividad	400, 401, 404, 422
GET	`/actividades`	`?estado=pendiente` o `completada`	200, actividades de los clientes de su cartera	400, 401
PATCH	`/actividades/:id`	`{ completada }`	200, la actividad	400, 401, 404, 422
GET	`/oportunidades`	`?etapa=`	200, resumen de sus oportunidades	400, 401
POST	`/oportunidades`	`{ cliente_id, titulo }`	201, detalle, en prospecto	400, 401, 422
GET	`/oportunidades/:id`		200, detalle	401, 404
PATCH	`/oportunidades/:id`	`{ titulo }`	200, detalle	400, 401, 404, 409, 422
PATCH	`/oportunidades/:id/etapa`	`{ etapa, motivo_perdida }`; motivo solo al perder	200, detalle	400, 401, 404, 409, 422
POST	`/oportunidades/:id/productos`	`{ producto_id, cantidad }`	201, detalle	400, 401, 404, 409, 422
PATCH	`/oportunidades/:id/productos/:lineaId`	`{ cantidad }`	200, detalle	400, 401, 404, 409, 422
DELETE	`/oportunidades/:id/productos/:lineaId`		204, sin cuerpo	401, 404, 409
GET	`/reportes/resumen`	`?mes=AAAA-MM`; por omisión, el mes actual de Guatemala	200, resumen del mes	400, 401

`:lineaId` es el `id` de la línea, el que aparece en `productos` dentro del detalle. Los campos que el contrato no lista (por ejemplo `vendedor_id`, `etapa` al crear o `precio_unitario`) nunca se usan.

Forma de las respuestas

Cliente (en una lista, cada elemento tiene esta forma)

```
{ "id": 1, "nombre": "Comercial El Quetzal", "nit": "4521817K",
  "telefono": "55551234", "correo": "compras@elquetzal.test" }
```

Inicio de sesión correcto

```
{ "token": "eyJhbGciOiJIUzI1NiIs...",
  "usuario": { "id": 4, "nombre": "Valeria López", "correo": "valeria@crm.test" } }
```

Detalle de una oportunidad (el resumen de la lista no trae productos ni historial)

```
{
  "id": 1,
  "cliente": { "id": 1, "nombre": "Comercial El Quetzal" },
  "titulo": "Renovación de planes",
  "etapa": "ganada",
  "probabilidad": 100,
  "total": 5650,
  "fecha_cierre": "2026-10-05T16:00:00.000Z",
  "motivo_perdida": null,
  "productos": [
    { "id": 1, "producto_id": 2, "nombre": "Plan avanzado",
      "cantidad": 2, "precio_unitario": 2600, "subtotal": 5200 },
    { "id": 2, "producto_id": 3, "nombre": "Instalación",
      "cantidad": 1, "precio_unitario": 450, "subtotal": 450 }
  ],
  "historial": [
    { "etapa_anterior": null, "etapa_nueva": "prospecto",
      "usuario": "Valeria López", "fecha": "2026-09-23T16:00:00.000Z" }
  ]
}
```

Actividad

```
{ "id": 2, "cliente": { "id": 2, "nombre": "Ferretería Xela" },
  "tipo": "tarea", "descripcion": "Enviar propuesta de capacitación",
  "vence_en": "2030-01-15T15:00:00.000Z", "completada": false, "vencida": false }
```

Resumen del mes

```
{ "mes": "2026-10", "ventas": 2, "total": 9250, "pronostico": 2512.88,
  "cartera": { "clientes": 3, "maximo": 10 } }
```

Errores: el primero para cualquier código; el segundo, para 422 (un mensaje por campo)

```
{ "error": "Su cartera ya tiene 10 clientes, el máximo permitido." }

{ "errores": {
    "nit": "Escriba un NIT válido (con su dígito verificador) o CF.",
    "telefono": "Escriba un teléfono de Guatemala de 8 dígitos." } }
```

Los ids de los ejemplos son ilustrativos. Los montos viajan como números: `9250` y `9250.00` son el mismo número en JSON.

Códigos de estado

Código	Cuándo se usa en este proyecto
200 OK	Lectura o modificación correcta
201 Created	Se creó un cliente, una oportunidad, una línea o una actividad
204 No Content	Se quitó un producto de una oportunidad; no hay cuerpo
400 Bad Request	La petición no se puede interpretar: JSON mal formado, falta el cuerpo, o un filtro o parámetro no existe
401 Unauthorized	Credenciales incorrectas; o falta el token, está alterado, venció o su cuenta ya no es válida

Código	Cuándo se usa en este proyecto
403 Forbidden	Credenciales correctas, pero la cuenta no puede usar la API: no es de vendedor o está desactivada
404 Not Found	El recurso no existe o es de otro vendedor
409 Conflict	La petición choca con el estado actual: cartera llena, cliente ya tomado, NIT o teléfono repetido, producto repetido, oportunidad cerrada
422 Unprocessable Content	La petición se entiende, pero los datos violan una regla
500 Internal Server Error	Error inesperado. El mensaje es genérico; el detalle queda en la consola del servidor

9. Requisitos

Cada requisito se cumple cuando se cumplen todos sus criterios. En su especificación los reescribirán con sus palabras y con criterios verificables.

Pieza	Códigos	Requisitos
Panel Laravel	L1 a L7	Esquema y semillas, cuentas, catálogo, clientes, equipos, reportes y validación
API	A1 a A8	Sesión, middleware, clientes, oportunidades, actividades, resumen, errores y documentación
SPA	F1 a F6	Sesión, cartera, embudo, actividades, resumen y calidad de la interfaz
Todo el sistema	S1 a S8	Seguridad (sección 10)
Proceso	P1 a P5	Especificación, plan, verificación y uso de IA (sección 12)

Panel Laravel

L1 · Esquema y semillas

- Las migraciones crean las tablas de la sección 6 con sus llaves foráneas, índices únicos y restricciones `CHECK`. La tabla `users` se extiende con una migración nueva, sin editar la que trae Laravel.
- `VerificacionSeeder` carga exactamente la semilla de la sección 11; `SectorSeeder` carga los datos de su sector.
- `php artisan migrate:fresh --seed` deja la base lista con las dos semillas, sin pasos manuales, y `php artisan migrate:fresh --seeder=VerificacionSeeder` la deja solo con la de verificación.

L2 · Cuentas y roles

- Inicio y cierre de sesión con Breeze (stack Blade). La página de registro y su ruta no existen.
- El administrador crea y edita cuentas: nombre, correo, contraseña, rol, supervisor (solo para vendedores, y debe ser un supervisor activo) y estado.
- Las cuentas no se borran: se desactivan. El administrador no puede desactivarse ni quitarse el rol a sí mismo.
- Un vendedor no puede iniciar sesión en el panel: ve un mensaje que lo envía a la SPA. Una cuenta desactivada no inicia sesión y, si tenía una sesión abierta, la pierde en su siguiente petición.

L3 · Catálogo

- El administrador agrega y edita productos; no los borra, los desactiva.
- El precio se valida mayor que cero y con dos decimales como máximo.

- Cambiar un precio no modifica ninguna oportunidad existente ni ningún reporte.

L4 · Clientes y asignación

- El administrador y el supervisor ven la lista de clientes que les corresponde, con búsqueda por nombre o NIT y filtro por vendedor y por "sin asignar".
- Crean y editan clientes con las reglas de R4: el NIT y el teléfono se normalizan antes de comprobar que no se repitan.
- Asignan un cliente a un vendedor activo, lo reasignan o lo liberan, respetando la cartera máxima de R3 y lo que su especificación decidió.
- El administrador elimina un cliente solo si no tiene oportunidades.

L5 · Equipos y visibilidad

- El supervisor ve únicamente a su equipo, a los clientes de su equipo y a los disponibles. Un cliente de otro equipo responde 404, aunque escriba su dirección a mano.
- Solo asigna a vendedores activos de su equipo.
- Tiene una página de su equipo con la cartera de cada vendedor (cuántos de 10) y sus ventas del mes.

L6 · Oportunidades y reportes

- El administrador ve todas las oportunidades y el supervisor las de su equipo, con filtro por etapa y por vendedor, el total de cada una y la fecha de cierre en hora de Guatemala. En el panel son de solo lectura: las trabaja el vendedor.
- El reporte del mes, con selector de mes, muestra ventas, total vendido y pronóstico, por vendedor y por producto, con las reglas de R7. El supervisor ve su equipo; el administrador, a todos.
- Con la semilla de verificación, el reporte coincide con las cifras de control de la sección 11.

L7 · Validación en el servidor

- Toda entrada se valida en el servidor (FormRequest o `validate`) con mensajes en español que tratan de usted. Si falla, el formulario vuelve con los datos escritos y el mensaje junto a cada campo.
- Las reglas de R3 y R4 dan el mismo resultado en el panel y en la API. Si las dos puertas validan distinto, el proyecto tiene un defecto aunque cada una funcione por separado.

API REST

A1 · Inicio de sesión con JWT

- `POST /api/auth/login` verifica la contraseña contra la tabla `users`.
- Un vendedor activo recibe 200 con `{ token, usuario }`. Un administrador, un supervisor o una cuenta desactivada, con la contraseña correcta, reciben 403 y ningún token.
- Correo inexistente y contraseña incorrecta responden 401 con exactamente la misma respuesta.
- El token es un JWT firmado con HS256, con el id del usuario en `sub` y una expiración configurable en `.env`.

> **Use bcryptjs, no bcrypt**
>
> PHP escribe sus hashes con el prefijo `$2y$`. El paquete `bcryptjs` los compara bien. El paquete `bcrypt` (nativo) responde `false` con la contraseña correcta, sin mostrar ningún error. Si su inicio de sesión falla siempre, revise primero esto.

A2 · Middleware de autenticación

- Todas las rutas, salvo el inicio de sesión, exigen `Authorization: Bearer <token>`.
- Sin token, o con un token alterado, vencido o firmado con otro algoritmo, la respuesta es 401.
- En cada petición se vuelve a leer al usuario en la base: si fue desactivado o ya no es vendedor, su token deja de servir.

A3 · Clientes y cartera

- Implementa los endpoints de clientes del contrato con R2, R3 y R4.
- Tomar un cliente es atómico: si diez vendedores intentan tomar el mismo cliente a la vez, exactamente uno lo consigue.
- La cartera máxima se respeta aunque lleguen varias altas o tomas simultáneas del mismo vendedor.

A4 · Oportunidades y productos

- Implementa los endpoints de oportunidades del contrato con R5 y R6.
- El precio de cada línea se copia del catálogo y el total lo calcula el servidor.
- El cambio de etapa, su registro en el historial y la fecha de cierre ocurren en una sola transacción; dos cambios simultáneos sobre la misma oportunidad no pueden dejar dos registros en el historial.

A5 · Actividades

- Implementa los endpoints de actividades del contrato con R2 y R8.
- `vence_en` se recibe en ISO 8601 con zona horaria y se devuelve en UTC; `vencida` se calcula al responder.

A6 · Resumen del mes

- `GET /api/reportes/resumen` calcula ventas, total y pronóstico con SQL, con las reglas de R7, y devuelve números.
- Con la semilla de verificación coincide con las cifras de control de la sección 11.

A7 · Errores, códigos y formato

- Cada respuesta usa el código de la sección 8 y siempre es JSON, nunca una página HTML.
- Los errores de validación responden 422 con un mensaje por campo; los demás errores, `{ error }`.
- Un error inesperado responde 500 con un mensaje genérico, y el detalle queda solo en la consola del servidor.

A8 · Documentación

- El README tiene la tabla de endpoints y los pasos exactos para levantar las tres piezas desde cero.
- La carpeta `docs/` incluye su colección de Postman: al menos una petición por endpoint, los casos de error principales y la prueba de su regla propia.

SPA

F1 · Sesión

- Hay una vista de inicio de sesión. Si la cuenta no es de vendedor, la SPA muestra el mensaje del 403.
- El token se guarda al entrar y se borra al cerrar sesión. LocalStorage solo guarda la sesión.
- Si cualquier petición responde 401 en plena sesión, la SPA vuelve al inicio de sesión con un aviso.

F2 · Cartera

- Lista de sus clientes con búsqueda en el servidor, y cuántos tiene de 10.
- Alta de cliente: los errores del servidor aparecen junto a cada campo y lo escrito se conserva.
- Lista de clientes disponibles con la acción de tomarlos. Si otro vendedor lo tomó antes, la SPA muestra el mensaje del 409.

F3 · Embudo

- Tablero con una columna por etapa abierta, y la lista de oportunidades cerradas.
- Detalle de cada oportunidad: productos con cantidad, precio guardado y subtotal; el total; y el historial con fecha y hora de Guatemala.
- Agregar productos del catálogo, cambiar cantidades, quitar productos y cambiar de etapa según las transiciones de su especificación. Perder pide el motivo.

F4 · Actividades

- Lista de pendientes y completadas; las vencidas se distinguen.
- Nueva actividad con fecha y hora local, que se envía en ISO 8601 con zona horaria. Marcar como completada.

F5 · Resumen del mes

- Selector de mes; ventas, total vendido, pronóstico y cartera, con montos en quetzales.
- Las fechas se muestran en hora de Guatemala, sin importar la zona horaria de la computadora.

F6 · Calidad de la interfaz

- Se aplican el patrón estado → render y la delegación de eventos.
- Un indicador de carga se muestra mientras hay peticiones en curso. Si la API está apagada, el usuario ve un mensaje que lo explica, no una pantalla vacía.
- Todo texto que viene de la API se escapa antes de insertarlo como HTML.
- Se usa en un teléfono sin desplazamiento horizontal a 375 píxeles de ancho.

10. Seguridad: lo que no se negocia

Un proyecto que falla en cualquiera de estos puntos tiene un defecto grave, aunque todo lo demás funcione.

Código	Regla	Cómo se comprueba
S1	Solo Laravel genera hashes de contraseñas; la API solo los verifica. Ninguna contraseña en texto plano en la base; en el código, solo la contraseña pública de las semillas.	Revisión de `users` y del código
S2	Los secretos no entran al repositorio: `.env` está en `.gitignore` en `laravel/` y en `api/`, y el repositorio incluye `.env.example` sin valores reales. `JWT_SECRET` tiene al menos 32 caracteres aleatorios.	Historial de Git. Borrar un `.env` en un commit posterior no lo saca del historial: si se filtró, cambie el secreto
S3	Ningún dato del usuario se concatena en el SQL: la API usa parámetros (`?`) y el panel usa Eloquent o enlaces de parámetros, nunca `DB::raw` con datos del usuario.	Revisión de código y prueba de inyección en la búsqueda
S4	Las respuestas no exponen hashes, llaves internas ni datos de otro vendedor (R8).	Respuestas en Postman
S5	Lo ajeno responde 404 en las dos puertas: un vendedor con lo de otro vendedor en la API, un supervisor con lo de otro equipo en el panel.	Valeria intenta leer y modificar un cliente de Diego; Sergio intenta editar un cliente del equipo de Sandra

Código	Regla	Cómo se comprueba
S6	CORS acepta solo el origen exacto de la SPA (por ejemplo, `http://127.0.0.1:5500`), configurado en `.env`. Nunca `*`.	Una página servida desde otro origen no puede llamar a la API
S7	La SPA escapa todo texto antes de insertarlo con `innerHTML`, o usa `textContent`. En Blade nunca se usa `{!! !!}` con datos del usuario.	Un cliente llamado `<img src=x onerror=alert(1)>` se muestra como texto
S8	Las reglas aguantan peticiones simultáneas: tomar un cliente, la cartera máxima, el cambio de etapa y los duplicados. Para los duplicados, el árbitro final es el índice único de la base: una consulta previa, sola, no alcanza.	Peticiones simultáneas contra la API

> **Por qué S7 importa más en este proyecto**
>
> La SPA guarda el token en LocalStorage. Cualquier script que logre ejecutarse en la página puede leerlo y actuar en nombre del vendedor. Escapar el texto es lo que impide que el nombre de un cliente se convierta en código.

11. Semilla de verificación

`VerificacionSeeder` carga exactamente estos datos. Es la misma para todas las parejas: las cifras de control y las pruebas del catedrático dependen de ella. Las fechas están en UTC y la contraseña de todas las cuentas es `Ventas2026`, pública a propósito: puede aparecer en las semillas, en el README y en su colección de Postman, y en ningún otro lugar del repositorio.

Cuentas

Nombre	Correo	Rol	Supervisor	Activa
Alicia Morales	`admin@crm.test`	administrador		sí
Sergio Ruiz	`sergio@crm.test`	supervisor		sí
Sandra Pérez	`sandra@crm.test`	supervisor		sí
Valeria López	`valeria@crm.test`	vendedor	Sergio Ruiz	sí
Diego Castillo	`diego@crm.test`	vendedor	Sergio Ruiz	sí
Carlos Ajú	`carlos@crm.test`	vendedor	Sandra Pérez	sí
Iván Toj	`ivan@crm.test`	vendedor	Sandra Pérez	**no**

Productos

Producto	Precio	Activo
Plan básico	Q1,200.00	sí
Plan avanzado	Q2,750.50	sí
Instalación	Q450.00	sí
Soporte anual	Q3,600.00	sí
Licencia anterior	Q999.99	**no**

Clientes

El NIT se muestra como se guarda: compacto, o CF para consumidor final (vacío en la base).

Vendedor	Cliente	NIT	Teléfono	Correo
Valeria López	Comercial El Quetzal	4521817K	55551234	compras@elquetzal.test
	Ferretería Xela	98765434	77612345	
	María Coyoy	CF	41234567	
Diego Castillo	Farmacia Los Altos	34567895	77634567	gerencia@losaltos.test
	Luis Tzul	CF	52345678	
Carlos Ajú	Abarrotes San Juan	12345679	77651001	
	Panadería La Esperanza	CF	30001002	
	Taller El Volcán	22233342	77651003	
	Ana Xicará	CF	30001004	
	Librería Cantel	55566677	77651005	

Vendedor	Cliente	NIT	Teléfono	Correo
	Pedro Batz	CF	30001006	
	Tienda La Bendición	CF	30001007	
	Distribuidora Occidente	77788893	77651008	
	Rosa Tax	CF	30001009	
Iván Toj	Hotel Los Pinos	66655544	77661001	
Sin asignar (disponibles)	Agencia Cuatro Caminos	CF	30002001	
	Café Zunil	CF	30002002	
	Clínica San Rafael	8889990K	77672003	
	Constructora Altiplano	44433328	77672004	
	Escuela Futuro	CF	30002005	
	Gimnasio Fuerza	CF	30002006	

Oportunidades

Los productos se indican como cantidad × precio guardado en la línea. Observe que Renovación de planes guardó el Plan avanzado a Q2,600.00: así costaba cuando se cotizó.

Título	Cliente	Vendedor	Etapa	Cierre (UTC)	Productos
Renovación de planes	Comercial El Quetzal	Valeria	ganada	2026-10-05 16:00	Plan avanzado 2 × Q2,600.00; Instalación 1 × Q450.00
Equipo para sucursal	Ferretería Xela	Valeria	ganada	2026-11-01 03:30	Plan básico 3 × Q1,200.00
Soporte del hogar	María Coyoy	Valeria	ganada	2026-11-01 06:30	Soporte anual 1 × Q3,600.00
Ampliación a bodega	Comercial El Quetzal	Valeria	negociación		Plan avanzado 1 × Q2,750.50
Capacitación de personal	Ferretería Xela	Valeria	propuesta		Instalación 2 × Q450.00
Plan familiar	María Coyoy	Valeria	perdida en propuesta; motivo: "Eligió a la competencia"	2026-10-20 18:00	Plan básico 1 × Q1,200.00; Licencia anterior 1 × Q999.99
Seguimiento anual	Comercial El Quetzal	Valeria	prospecto		sin productos
Licencias de caja	Farmacia Los Altos	Diego	ganada	2026-10-10 15:00	Plan básico 2 × Q1,200.00; Soporte anual 1 × Q3,600.00
Mantenimiento	Luis Tzul	Diego	calificado		Instalación 1 × Q450.00
Plan anual	Abarrotes San Juan	Carlos	ganada	2026-10-15 20:00	Soporte anual 2 × Q3,600.00
Cotización de equipo	Taller El Volcán	Carlos	negociación		Plan avanzado 2 × Q2,750.50

Historial de la semilla. Cada oportunidad tiene un registro por cada etapa que recorrió, hechos por su vendedor: el de creación (etapa anterior vacía, etapa nueva prospecto) y uno por cada avance hasta la etapa actual. Las ganadas pasaron por las cuatro etapas abiertas; Plan familiar se perdió desde propuesta. En total son 43 registros. Sugerencia de fechas: tres días entre registros, terminando en la fecha de cierre, o el 2 de octubre de 2026 a las 16:00 UTC si sigue abierta.

Actividades

Cliente	Vendedor	Tipo	Descripción	Vence (UTC)	Completada
Comercial El Quetzal	Valeria	llamada	Confirmar fecha de instalación	2026-10-08 15:00	sí
Ferretería Xela	Valeria	tarea	Enviar propuesta de capacitación	2030-01-15 15:00	no
María Coyoy	Valeria	reunion	Presentar opciones de soporte	2026-09-01 15:00	no
Farmacia Los Altos	Diego	llamada	Renovación de licencias	2030-02-01 15:00	no

Cifras de control

Con la semilla de verificación recién cargada, su sistema debe mostrar estas cifras, en el panel y en la API:

Quién	Mes	Ventas	Total vendido	Pronóstico	Cartera
Valeria López	octubre 2026	2	Q9,250.00	Q2,512.88	3 de 10
Valeria López	noviembre 2026	1	Q3,600.00	Q2,512.88	3 de 10
Diego Castillo	octubre 2026	1	Q6,000.00	Q112.50	2 de 10
Carlos Ajú	octubre 2026	1	Q7,200.00	Q4,125.75	9 de 10
Equipo de Sergio Ruiz	octubre 2026	3	Q15,250.00	Q2,625.38	
Equipo de Sandra Pérez	octubre 2026	1	Q7,200.00	Q4,125.75	
Toda la empresa	octubre 2026	4	Q22,450.00	Q6,751.13	

Producto · octubre 2026	Unidades vendidas	Total
Plan básico	5	Q6,000.00
Plan avanzado	2	Q5,200.00
Instalación	1	Q450.00
Soporte anual	3	Q10,800.00

> **Si sus cifras no coinciden**
>
> Si a Valeria le salen **Q5,650.00** en octubre, está agrupando los meses en UTC: la venta del 1 de noviembre a las 03:30 UTC ocurrió el 31 de octubre a las 21:30 en Guatemala.
>
> Si le salen **Q9,551.00**, está multiplicando por el precio actual del catálogo y no por el precio guardado en la línea.
>
> Si la API devuelve `"9250.00"` entre comillas, mysql2 le está entregando el DECIMAL como texto y no como número.

12. Proceso: desarrollo guiado por especificaciones con IA

El proyecto sigue las cuatro fases que trabajaron en clase: **especificación, plan, implementación y verificación**. Las plantillas de los documentos están en el aula virtual. Si usan una herramienta de desarrollo guiado por especificaciones que genera sus propios archivos (por ejemplo, Spec Kit), pueden usarla: lo que se exige es el contenido, no el nombre del archivo.

Código	Qué se entrega	Detalle
P1	`docs/especificacion.md`, antes del código	Qué hace el sistema y cómo se comprueba: su sector y su regla propia, los requisitos con criterios Dado / Cuando / Entonces, el contrato completo, el modelo de datos con el lugar donde se cumple cada regla y las decisiones de la sección 7 con la alternativa descartada
P2	`docs/plan.md`	Arquitectura, estructura, fases y tareas con responsable, y qué se le pedirá al agente en cada fase. Se actualiza antes de cada fase
P3	`docs/verificacion.md`	Matriz que une cada criterio de aceptación con su evidencia, el resultado de las pruebas de seguridad (incluidas las de peticiones simultáneas) y las cifras de control obtenidas
P4	`CLAUDE.md` o `AGENTS.md` en la raíz	El archivo de contexto de su agente, con las reglas del proyecto que el agente debe respetar
P5	Bitácora de IA, en `verificacion.md`	Al menos tres casos en que el agente propuso algo incorrecto, inseguro o distinto de la especificación: cómo lo detectaron y el commit que lo corrigió

Primero la especificación

La especificación describe **qué** hace el sistema y se puede escribir antes de dominar Laravel o Express. Cuando la tengan, márquenla en Git con la etiqueta `spec-v1`. El historial debe mostrar que el código de implementación llegó después de esa etiqueta: un agente que programa sin especificación decide por ustedes todo lo que ustedes no decidieron.

Mientras escriben la especificación, preparen el entorno: instalen las herramientas, creen el proyecto Laravel con Breeze y conecten la base MySQL.

Un cambio de requisito

En algún momento del proyecto, el catedrático publicará en el aula virtual un cambio de requisito, como sucede en cualquier proyecto real. No se rehace el proyecto: se actualiza la especificación (etiqueta `spec-v2`), se planifica el cambio y se implementa sin romper lo que ya funcionaba. Las migraciones que ya existen no se editan: el cambio llega con migraciones nuevas.

El historial también se evalúa

Commits de ambos integrantes, repartidos en el tiempo, con mensajes que digan qué cambió. Un historial con dos commits el último día no demuestra ningún proceso, por bueno que sea el resultado.

> **Sobre la inteligencia artificial**
>
> Está permitida y se espera que la usen. Lo que se evalúa es que entiendan, revisen y puedan modificar lo que el agente produjo. En la defensa se les pedirá cambiar código en vivo y explicar partes que escribió su compañero o el agente.

13. Estructura del repositorio

```
crm-<su-sector>/
├── README.md
├── CLAUDE.md  o  AGENTS.md
├── .gitignore
├── docs/
│   ├── especificacion.md
│   ├── plan.md
│   ├── verificacion.md
│   └── api.postman_collection.json
├── laravel/            el panel (sin vendor/ ni node_modules/)
├── api/
│   ├── package.json
│   ├── .env.example
│   └── src/
└── frontend/
    ├── index.html
    ├── css/
    └── js/
```

Orden de trabajo sugerido

1. Sector, regla propia y especificación (P1).
2. Laravel: migraciones, semillas, cuentas y roles (L1, L2), y después catálogo, clientes, equipos y reportes (L3 a L7), a medida que avanzan las clases de Laravel.
3. API: inicio de sesión y middleware (A1, A2), y después clientes, oportunidades, actividades y resumen (A3 a A7), a medida que avanzan las clases de Node.js.
4. SPA contra la API (F1 a F6).
5. Verificación, documentación y ensayo de la defensa (A8, P3 y P5).

Pueden adelantar piezas en paralelo, pero no dejen la integración para el final: la primera petición real de la SPA a su API es donde aparecen los problemas de CORS, y la primera venta ganada en la SPA es donde se descubre si el reporte del panel cuenta bien.

14. Entrega y defensa

Entrega

- Hay una sola entrega, la semana anterior al examen final: por el aula virtual se envía el enlace al repositorio y el identificador del commit de la etiqueta `entrega-final`, que se obtiene con `git rev-parse entrega-final`.
- El repositorio debe ser accesible para el catedrático: público, o privado con el catedrático como colaborador.
- Lo que no esté en el commit de la etiqueta `entrega-final` no se considera entregado.

Defensa oral

- Es oral, de 15 minutos por pareja.
- La pareja llega con las tres piezas corriendo en una computadora, la base cargada solo con la semilla de verificación (`php artisan migrate:fresh --seeder=VerificacionSeeder`), Postman abierto y el repositorio en la etiqueta `entrega-final`.
- El catedrático ejecuta sus propias pruebas sobre la API y el panel, pide explicar decisiones de la especificación y pide modificar código en vivo.

- Cada integrante responde solo; el compañero no interviene. Las preguntas pueden ser sobre cualquier parte del proyecto, incluida la que escribió su compañero o el agente.
- "No sé por qué funciona; lo hizo la IA" equivale a no haberlo hecho.

Retos opcionales

Suman nota. Lo que agreguen va además de lo pedido: un reto no puede romper el contrato ni ningún requisito.

- **Resumen de un cliente con un modelo de lenguaje.** Un endpoint nuevo de la API arma, con el historial y las actividades del cliente, un resumen en lenguaje natural para el vendedor. La clave del proveedor vive en `api/.env` y nunca llega a la SPA; el endpoint respeta la propiedad de R2 y maneja con un mensaje claro que el proveedor falle o tarde.
- Desplegar las tres piezas en un servicio en la nube.
- Pruebas automatizadas: `node:test` en la API o pruebas de Laravel en el panel.
- Limitar los intentos fallidos de inicio de sesión en la API.
- Exportar el reporte del mes del panel a CSV.